Assignment 9

Object Orientation

Spring 2019

This assignment is new in 2019. You can **not** re-use a grade from previous years.

1 JavaFX

JavaFX is a framework for creating graphical user interfaces in Java.

2 Learning Goals

In this exercise you use JavaFX to create an application with a graphical user interface. After doing this exercise you should be able to:

- Use panes to automatically layout GUI elements
- Use properties to automatically update GUI elements

3 Problem Sketch

In this assignment you implement a GUI for a pie chart generator. To keep it simple, the pie chart has a fixed number of four segments. You also do not have to draw the pie chart, but only display the proportions of the segments.

Figure 1 shows a screenshot of the pie chart generator. On the left side are four input fields that only accept positive integers. On the right side are four labels that display the proportion of each integer relative to the total sum. When the user changes one of the input values, all outputs should update automatically.

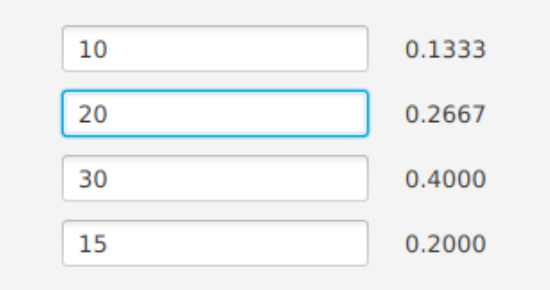

Figure 1: Screenshot of the simple pie chart generator.

3.1 Using Panes

In JavaFX, a `Pane` manages the layout of its child elements. There are different panes that arrange their children in different ways. Panes can be nested. In this assignment you should use a `GridPane`.

A `GridPane` lays its children out in a tabular fashion. The screenshot in fig. 1 is made with the following settings.

```
GridPane root = new GridPane();
root.setAlignment(Pos.CENTER);
root.setHgap(20);
root.setVgap(10);
```

3.2 Validating Input

JavaFX text fields support a callback mechanism that lets you register a function that is called every time the user changes the input. To make it easy, we provide the code you should use.

```
TextField textField = new TextField();
textField.textProperty().addListener(
  new ChangeListener<String>() {
  @Override
  public void changed(
    ObservableValue<? extends String> observable,
    String oldValue, String newValue) {
    if (!newValue.matches("[1-9]\\d{0,3}")) {
      textField.setText(oldValue);
    } } });
```

This function checks if the string in the text field matches a simple regular expression. If it does not match, the old value is put back. The given regular expression ensures a positive integer of at most 4 digits.

3.3 Using Properties

When the user changes one of the input fields, the output fields should update automatically. For this you should use `Properties`.

The input and output fields have text properties, but you need to do some arithmetic with their values. For this you should build a network of properties that depend on each other and perform the necessary conversions.

For every input field you should create a `SimpleIntegerProperty` that is synchronized with the text of the input field. You can bind it to the text using `Property.bindBidirectional()` and `NumberStringConverter`. This gives you a translation and update from a string property to an integer property.

Next, you should create a `SimpleIntegerProperty` called `sum` that holds the sum of all the text field's integer properties. How to keep the sum updated is part of the exercise. Hint: Create a new intermediate sum for each input field that depends on the previous intermediate sum.

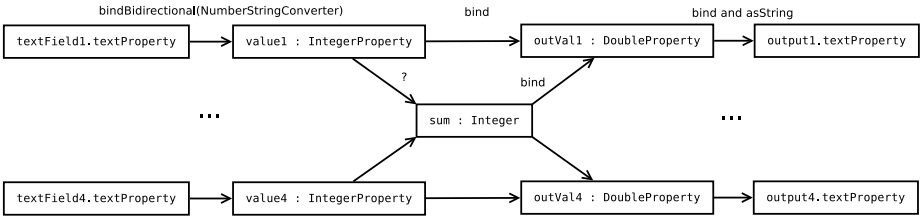

Figure 2: The properties and their dependencies.

You should create a `SimpleDoubleProperty` for each output field, that is bound to the ratio of the corresponding input property and the sum. This requires a conversion from integer to double in the binding. The ratio is a real number between 0 and 1, but both sum and input properties are integers. If you divide an integer by a bigger integer, it will be rounded to 0. You have to convert the integer to double before the division. This can be done by using `IntegerProperty.add(0f)`.

The final step is feeding each double property into its respective output label. This can be done with `DoubleProperty.asString`, which takes a format string as argument.

Figure 2 shows the properties in the system. The arrows denote dependencies and are labelled with the way the properties should be updated.

4 Your Tasks

1. Create a new JavaFX project in NetBeans.
2. Make sure to use Java 8, JDK 1.8
3. Create panes for the layout.
4. Add TextFields and Labels to the panes.
5. Create properties as necessary and implement the update mechanics.

5 Optional Extra Challenge: Draw The Pie Chart

If you want, you can try drawing the pie chart. Instead of a `GridPane`, you should use a `BorderPane`. Put the input fields in a `VBox`, which you put in the left area of the BorderPane. Put the pie chart drawing in a simple `Pane`, which you put in the center area of the `BorderPane`. Create `Arcs` with the center and radius bound to the dimensions of the pane. The start and end angle should be bound to the corresponding output properties. When the user changes an input field, the drawing should update automatically.

6 Optional Extra Extra Challenge: Variable Number of Segments

Add a plus and a minus button to dynamically add and remove segments. You can put those buttons in the bottom pane of the BorderPane.

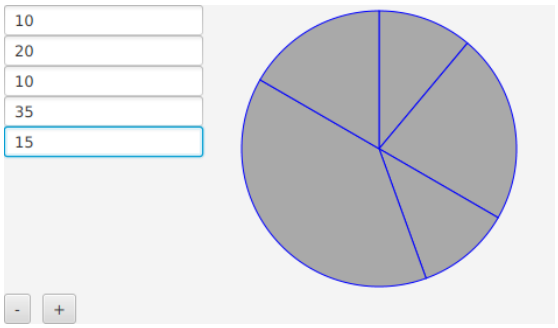

Figure 3: Optional Extra Challenge: draw the pie chart with variable number of segments.

7 Submit Your Project

To submit your project, follow these steps.

1. Use the **project export** feature of NetBeans to create a zip file of your entire project: File → Export Project → To ZIP.
2. **Submit this zip file on Brightspace**. Do not submit individual Java files. Do not submit any other archive format. Only one person in your group has to submit it. Submit your project before the deadline, which can be found on Brightspace.